ASTANA IT UNIVERSITY — DEPARTMENT OF COMPUTER ENGINEERING
COURSE: ShP-2216 SOFTWARE DESIGN PATTERNS | RK2 LABORATORY ASSESSMENT

Visitor Pattern — Advanced In-Memory File System

Author: Muratbek Akyl **Group:** SE 2523 **Stack:** Java 17 | Maven | JUnit 5 **Repo:** github.com/muratbek-akyl/SDP_Assignment6

1. Purpose & Architectural Design Choices

This project implements an in-memory hierarchical file system utilizing the **Visitor Behavioral Design Pattern** (Gang of Four) to strictly decouple operational algorithms from the underlying composite data structure. In conventional hierarchical models, adding operations like size accumulation, multi-criteria filtering, or tree statistics causes class bloat, violates the Single Responsibility Principle, and couples nodes to transient domain logic. The Visitor pattern solves this by externalizing operations into specialized visitor objects without modifying the elements.

Double Dispatch Protocol (R2 & Engineering Constraint):
Double dispatch routes execution through two coordinated polymorphic dispatches:
• **Step 1 (Dynamic Dispatch on Element):** Client invokes element.accept(visitor). At runtime, the JVM dispatches to the concrete element instance (FileNode or DirectoryNode).
• **Step 2 (Static Overload Resolution + Dynamic Dispatch on Visitor):** The element invokes visitor.visit(this). Because this has compile-time static type FileNode or DirectoryNode, the Java compiler statically resolves the exact overloaded method signature without any runtime type discrimination (**zero instanceof, zero getClass(), zero casts**).

2. Traversal Ownership & Architectural Trade-offs

Traversal Responsibility (R7): We selected **Visitor-Owned Traversal (Internal Traversal)** over element-driven traversal or external walkers:

- **Contextual State Isolation:** Different visitor operations require distinct transient state machines. SearchVisitor maintains a Deque<String> path stack to deterministically generate root-relative paths, while StatisticsVisitor tracks hierarchical depth levels. Placing traversal inside the visitor encapsulates contextual stacks without polluting Element.
- **Single Responsibility Principle:** DirectoryNode and FileNode remain lightweight composite containers managing child containment and byte sizes, free of traversal orchestration logic.
- **Guaranteed Reachability:** Every reachable node in the hierarchy is visited exactly once per execution. Both nested hierarchies and empty directories (depth and directory count) are comprehensively supported.

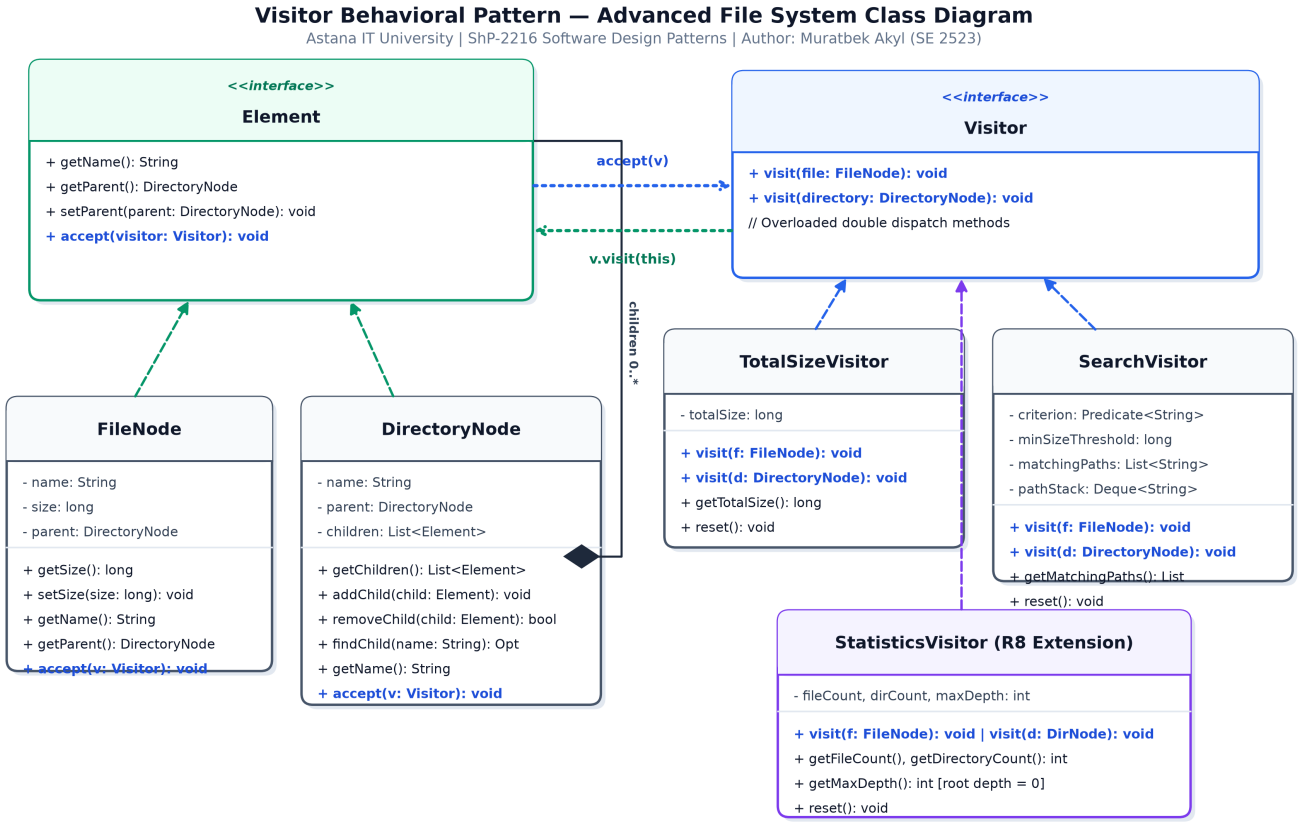
Architectural Trade-off of the Visitor Pattern:
• **Adding New Operations (Extremely Cheap):** Fully conforms to the *Open/Closed Principle (OCP)*. Introducing StatisticsVisitor (R8) required zero modifications to Element, FileNode, or DirectoryNode.
• **Adding New Element Types (Costly):** Introducing a new element (e.g. SymlinkNode) breaks the Visitor interface contract, requiring a new visit(SymlinkNode) signature and forcing manual updates across all existing concrete visitors.

3. Five Concrete Clean Code Justifications (Section 6)

1. **Meaningful Naming (SearchVisitor.getMatchingPaths(), TotalSizeVisitor.getTotalSize()):** Methods and variables employ self-describing domain terms that communicate operational intent unambiguously without inline commentary.
2. **Single Responsibility Principle (FileNode vs. StatisticsVisitor):** Domain entities model structural hierarchy invariants exclusively, while computational algorithms (size accumulation, query filtering, metrics) are isolated in dedicated visitor classes.
3. **Small Methods & Minimal Complexity (SearchVisitor.buildRelativePath()):** Methods are concise (under 12 lines) and perform a single logical task with zero nested looping, ensuring high readability and easy testability.
4. **Encapsulation & Defensive Invariants (DirectoryNode.getChildren()):** Internal child collections are wrapped in Collections.unmodifiableList(). All modifications are forced through mutating methods enforcing tree integrity (R6).
5. **Elimination of Type-Checking Chains (Double Dispatch in Element.accept()):** Replaced fragile if-else ladders and instanceof checks with polymorphic double dispatch, adhering strictly to engineering constraints.

4. System Architecture & UML Class Diagram

The class diagram below illustrates the complete object model, highlighting the separation between the Composite Element Hierarchy (left) and the Visitor Hierarchy (right), bridged by the bidirectional double dispatch interaction.



Key Architectural Components:

- **Element Interface:** Common abstraction declaring getName(), getParent(), setParent(), and accept(Visitor).
- **FileNode (Leaf):** Holds immutable name and non-negative byte size; dispatches to visitor.visit(this).
- **DirectoryNode (Composite):** Encapsulates unmodifiable child collection; guards invariants (non-null, no duplicate siblings, single-parent, cycle detection); dispatches to visitor.visit(this).
- **Visitor Interface:** Defines overloaded visit(FileNode) and visit(DirectoryNode) contracts.
- **Concrete Visitors:** TotalSizeVisitor (R3 size aggregator), SearchVisitor (R4 multi-constraint deterministic query), StatisticsVisitor (R5 metrics collector, R8 extension boundary), and TreePrinterVisitor (ASCII visualization).

5. Tree Integrity & Engineering Invariants (R6)

To guarantee hierarchical stability and prevent corrupted data states, all mutating operations enforce strict invariant checks:

Integrity Constraint	Enforcement Location	Behavioral Outcome & Exception
Non-Negative Size	`FileNode(name, size)` `setSize(long)`	Rejects negative values immediately with IllegalArgumentException; preserves previous state.
Duplicate Sibling Names	`DirectoryNode.addChild()`	Case-insensitively checks siblings; rejects identical names with IllegalArgumentException.
Single-Parent Ownership	`DirectoryNode.addChild()`	Prevents node adoption if getParent() != null; throws IllegalStateException unless detached.
Cycle & Self-Nesting	`DirectoryNode.addChild()`	Traverses ancestor parent chains backwards; detects self-attachments and ancestor cycles, throwing IllegalStateException.

6. Controlled State Management & Determinism (Section 4)

- **Controlled State (T7):** Visitors support an explicit reset() method clearing internal accumulators (totalSize = 0, matchingPaths.clear(), pathStack.clear(), depth counters). Re-running a visitor produces idempotent, leak-free results.
- **Deterministic Outputs (R4):** Search paths are constructed relative to the chosen root using / as the separator and sorted lexicographically via Collections.sort() before returning.

7. Automated Verification & Test Results (T1 – T8)

All tests execute automatically from project root using mvn test without IDE reliance. All 20 tests pass with zero failures:

ID	Test Class Name	Verification Scope & Asserted Targets	Status
T1	`DoubleDispatchTest`	Both element types reach correct overloaded visit methods via double dispatch; null visitor rejected.	**PASSED (2/2)**
T2	`TotalSizeVisitorTest`	Size accumulation across 4+ nesting levels is accurate; directories contribute exactly 0 bytes.	**PASSED (3/3)**
T3	`SearchVisitorTest`	Combines both filename criterion AND inclusive minimum size threshold (size >= threshold).	**PASSED (1/1)**
T4	`SearchVisitorTest`	Deterministic root-relative paths using '/' separator, sorted lexicographically; custom Predicates.	**PASSED (2/2)**
T5	`StatisticsVisitorTest`	Collects accurate file count, directory count (including root), and maximum depth (root depth = 0).	**PASSED (2/2)**
T6	`SearchVisitorTest` `StatisticsVisitorTest`	Empty directories, 0-byte files, and no-match search queries handled gracefully without error.	**PASSED (2/2)**
T7	`StatisticsVisitorTest`	Controlled state: invoking reset() ensures repeat executions do not leak or multiply state.	**PASSED (1/1)**
T8	`TreeIntegrityTest`	Rejects negative size, blank names, duplicate siblings, multi-parent, and cycles while valid nesting works.	**PASSED (7/7)**

8. Deterministic Application Execution & Sample Output

Verbatim execution transcript produced by kz.aitu.filesystem.Main demonstrating the full composite hierarchy, visitor operations, state resets, and tree integrity enforcement:

```
================================================================================
   ASTANA IT UNIVERSITY -- SOFTWARE DESIGN PATTERNS (ShP-2216)
   ASSIGNMENT 06: VISITOR PATTERN -- ADVANCED FILE SYSTEM
   Student: Muratbek Akyl | Group: SE 2523
================================================================================

[1] HIERARCHY STRUCTURE INITIALIZED:
+-- [aitu-project-root/]
    |-- pom.xml (2048 bytes)
    |-- README.md (1536 bytes)
    +-- [src/]
        +-- [main/]
            +-- [java/]
                |-- Application.java (4096 bytes)
                |-- Config.java (1024 bytes)
            +-- [resources/]
                |-- application.properties (512 bytes)
                |-- schema.sql (3072 bytes)
        +-- [test/]
            +-- [java/]
                |-- ApplicationTest.java (2560 bytes)
    +-- [docs/]
        |-- architecture.pdf (1048576 bytes)
        |-- report.md (4096 bytes)
    +-- [logs/]
    +-- [data/]
        |-- export.csv (524288 bytes)

[2] TOTAL SIZE VISITATION (R3):
    Subtree Total Size: 1591808 bytes (1554.50 KB / 1.52 MB)

[3] SEARCH VISITATION WITH COMBINED CONSTRAINTS (R4):
    Query A [Criterion: contains '.java', Min Size >= 2000 B]:
      -> src/main/java/Application.java
      -> src/test/java/ApplicationTest.java
    Query B [Criterion: all names, Min Size >= 500,000 B]:
      -> data/export.csv
      -> docs/architecture.pdf
    Query C [Criterion: 'nonexistent_extension', Min Size >= 1 B]:
      (no matches found)

[4] STATISTICS VISITATION (R5 & R8):
    Total Files Visited       : 10
    Total Directories Visited : 10 (includes root directory)
    Maximum Tree Depth        : 4 (root depth = 0)

[5] CONTROLLED STATE & IDEMPOTENT RE-EXECUTION (T7):
    Re-running StatisticsVisitor after explicit reset()...
    After Reset -- Files: 10, Directories: 10, MaxDepth: 4 (No state leakage)

[6] TREE INTEGRITY ENFORCEMENT (R6):
    [PASS] Negative file size rejected: File size cannot be negative: -100
    [PASS] Duplicate sibling name rejected: Duplicate sibling name 'pom.xml' in directory 'aitu-project-root'
    [PASS] Multiple parent ownership rejected: Element 'pom.xml' already belongs to parent 'aitu-project-root'. Detach first.
    [PASS] Cycle detected and rejected: Cycle detected: cannot add ancestor 'aitu-project-root' to directory 'src'
================================================================================
```

9. Individual Defense Preparation Checklist & Submission Details

Oral Defense Protocol Checklist: 1. **Dispatch Trace:** Traced from element.accept(v) to v.visit(this). 2. **Automated Test Execution:** Run mvn test (20/20 tests passing in ~1.5s). 3. **Dynamic Query Mutation:** Demonstrated flexible Predicate/size filtering. 4. **Architectural Defense:** Articulated operation vs element extensibility trade-off.	**Repository & Build Information:** • **GitHub:** github.com/muratbek-akyl/SDP_Assignment6 • **Student:** Muratbek Akyl | Group: SE 2523 • **Test Command:** mvn clean test • **Run Command:** mvn compile exec:java • **Java Version:** Java 17 | Maven 3.9+